

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA Nº 07

NOMBRE COMPLETO: Robles Chávez Uri Shuhua Alexei

Nº de Cuenta: 320195318

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 05

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 10 de Octubre de 2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

Descripción

En esta parte de la práctica se implementaron fuentes de iluminación tipo SpotLight asociadas a objetos tridimensionales con movimiento y jerarquías.

El objetivo fue conseguir que la posición y dirección de cada fuente de luz dependieran de la transformación del objeto al que pertenece. De esta forma, cuando el Rover avanza o retrocede, su faro permanece unido al vehículo, mientras que las luces del avión acompañan sus movimientos verticales y longitudinales.

Para esto se utilizaron matrices padre que contienen las transformaciones generales de cada objeto y que posteriormente son utilizadas tanto para dibujar sus componentes como para calcular la posición y dirección de sus luces.

Desarrollo

Ejercicio 1. Spotlight frontal del Rover jerárquico

Para este ejercicio se añadió un Rover construido mediante una estructura jerárquica.

Se crearon diferentes mallas básicas para formar sus componentes, incluyendo cubos, cilindros y esferas. Estas piezas son utilizadas para construir la cabina, base, brazo articulado, pinza y las seis ruedas.

```
// agregado del rover
void CrearMallasRoverJerarquico(MeshModel* mallas)
{
    GLfloat cubo[] = {
        -0.5f, -0.5f, 0.5f, 0.0f, 0.0f, 0.0f, 0.0f, 1.0f,
        0.5f, -0.5f, 0.5f, 1.0f, 0.0f, 0.0f, 0.0f, 1.0f,
        0.5f, 0.5f, 0.5f, 1.0f, 1.0f, 0.0f, 0.0f, 1.0f,
        -0.5f, 0.5f, 0.5f, 0.0f, 1.0f, 0.0f, 0.0f, 1.0f,
        0.5f, -0.5f, -0.5f, 0.0f, 0.0f, 0.0f, 0.0f, -1.0f,
        -0.5f, -0.5f, -0.5f, 1.0f, 0.0f, 0.0f, 0.0f, -1.0f,
        -0.5f, 0.5f, -0.5f, 1.0f, 1.0f, 0.0f, 0.0f, -1.0f,
        0.5f, 0.5f, -0.5f, 0.0f, 1.0f, 0.0f, 0.0f, -1.0f,
        0.5f, -0.5f, 0.5f, 0.0f, 0.0f, 1.0f, 0.0f, 0.0f,
        0.5f, -0.5f, -0.5f, 1.0f, 0.0f, 1.0f, 0.0f, 0.0f,
        0.5f, 0.5f, -0.5f, 1.0f, 1.0f, 1.0f, 0.0f, 0.0f,
        0.5f, 0.5f, 0.5f, 0.0f, 1.0f, 1.0f, 0.0f, 0.0f,
        -0.5f, -0.5f, -0.5f, 0.0f, 0.0f, -1.0f, 0.0f, 0.0f,
        -0.5f, -0.5f, 0.5f, 1.0f, 0.0f, -1.0f, 0.0f, 0.0f,
        -0.5f, 0.5f, 0.5f, 1.0f, 1.0f, -1.0f, 0.0f, 0.0f,
        -0.5f, 0.5f, -0.5f, 0.0f, 1.0f, -1.0f, 0.0f, 0.0f,
        -0.5f, 0.5f, 0.5f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f,
        0.5f, 0.5f, 0.5f, 1.0f, 0.0f, 0.0f, 1.0f, 0.0f,
        0.5f, 0.5f, -0.5f, 1.0f, 1.0f, 0.0f, 1.0f, 0.0f,
        -0.5f, 0.5f, -0.5f, 0.0f, 1.0f, 0.0f, 1.0f, 0.0f,
        -0.5f, -0.5f, -0.5f, 0.0f, 0.0f, 0.0f, -1.0f, 0.0f,
        0.5f, -0.5f, -0.5f, 1.0f, 0.0f, 0.0f, -1.0f, 0.0f,
        0.5f, -0.5f, 0.5f, 1.0f, 1.0f, 0.0f, -1.0f, 0.0f,
        -0.5f, -0.5f, 0.5f, 0.0f, 1.0f, 0.0f, -1.0f, 0.0f,
    };
```

Figura 1. Creación de las mallas básicas utilizadas para construir el Rover mediante una estructura jerárquica.

Jerarquía del Rover

El Rover utiliza una matriz padre:

glm::mat4 matrizRover = glm::translate(glm::mat4(1.0f), posicionRover);

matrizRover = glm::scale(matrizRover, glm::vec3(0.45f));

Esta misma transformación general se utiliza como base para la carrocería, ruedas, brazo articulado y faro. Esto permite que todos los elementos permanezcan unidos cuando cambia la posición general del Rover.

Movimiento del Rover

El desplazamiento se realizó mediante las flechas del teclado:

↑ = avanzar

↓ = retroceder

El código obtiene el sentido del movimiento y modifica la coordenada X de posicionRover:

GLfloat avanceRover = (teclas[GLFW_KEY_UP] ? 1.0f : 0.0f) - (teclas[GLFW_KEY_DOWN] ? 1.0f : 0.0f);

GLfloat distanciaRover = avanceRover * 3.0f * dtRover;

posicionRover.x -= distanciaRover;

También se actualiza el giro de las ruedas de acuerdo con la distancia recorrida, consiguiendo que el movimiento del Rover sea visualmente coherente.

```
492     //carga de los tres grupos originales
493     GLint alineacionAvion = 4;
494     glGetIntegerv(GL_UNPACK_ALIGNMENT, &alineacionAvion);
495     glPixelStorei(GL_UNPACK_ALIGNMENT, 1);
496     avionCuerpo_M.LoadModel("Models/avion_p7_cuerpo.obj");
497     avionCara_M.LoadModel("Models/avion_p7_cara.obj");
498     avionAlas_M.LoadModel("Models/avion_p7_alas.obj");
499     glPixelStorei(GL_UNPACK_ALIGNMENT, alineacionAvion);
500     //inicializacion rover
501     MeshModel mallasRoverJerarquico[3];
502     CrearMallasRoverJerarquico(mallasRoverJerarquico);
503     GLfloat angulosRover[10] = {};
504     glm::vec3 posicionRover(8.0f, -1.0f, -6.0f);
505     const glm::vec3 posicionFaroLocal(-5.2f, 4.2f, -4.0f); // Frente, lado del brazo.
506     const glm::vec3 direccionFaroLocal = glm::normalize(glm::vec3(-1.0f, -0.8f, 0.0f)); // Hacia delante y al piso cercano.
507     double tiempoRoverAnterior = glfwGetTime();
508     glm::vec3 posicionAvion(-12.0f, 4.0f, -8.0f);
509     GLfloat giroAvion = 0.0f;
510     //fin de la inicializacion.
```

Figura 2. Desplazamiento del Rover mediante las teclas de dirección conservando su estructura jerárquica.

Spotlight frontal azul del Rover

Para el Rover se creó un spotlight de color azul:

spotLights[indiceFaroRover] = SpotLight (0.0f, 0.0f, 1.0f,0.05f, 2.0f,0.0f, 0.0f, 0.0f,-1.0f, -0.8f, 0.0f,1.0f, 0.05f, 0.01f,30.0f);

La dirección local del faro está orientada principalmente hacia -X, que en este Rover corresponde a su frente, con una pequeña inclinación hacia el piso.

La característica más importante es que la luz no utiliza una posición fija. Su posición mundial se calcula utilizando la misma matriz padre del Rover:

glm::vec3 posicionFaroMundo = glm::vec3(matrizRover * glm::vec4(posicionFaroLocal, 1.0f));

glm::vec3 direccionFaroMundo = glm::normalize(glm::mat3(matrizRover) * direccionFaroLocal);

y finalmente: spotLights[indiceFaroRover].SetFlash (posicionFaroMundo, direccionFaroMundo);

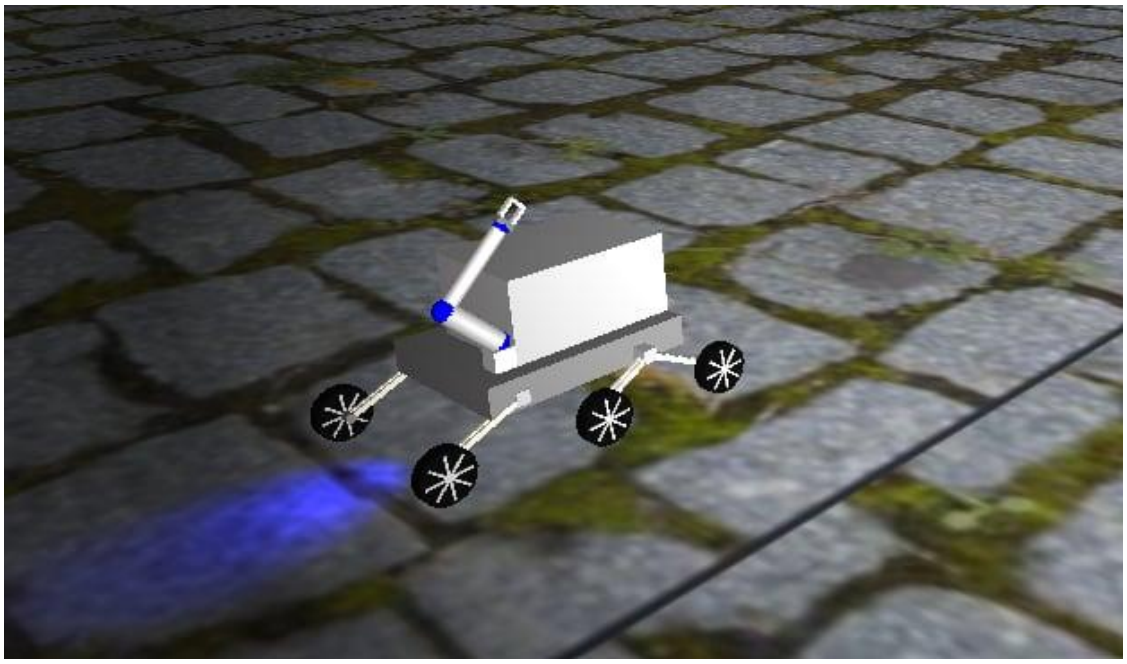


Figura 3. Spotlight azul colocado en la parte frontal del Rover e iluminando en su dirección de avance.

Ejercicio 2. Spotlight inferior del avión

Para los siguientes ejercicios se utilizó un avión compuesto por tres modelos:

- Model avionCuerpo_M;
- Model avionCara_M;
- Model avionAlas_M;

Estos modelos son cargados individualmente:

- avionCuerpo_M.LoadModel("Models/avion_p7_cuerpo.obj");
- avionCara_M.LoadModel("Models/avion_p7_cara.obj");
- avionAlas_M.LoadModel("Models/avion_p7_alas.obj");

pero todos utilizan posteriormente una misma matriz padre padreAvion, por lo que se comportan como partes de un mismo objeto jerárquico.

Movimiento vertical del avión

El avión puede subir y bajar mediante:

9 = subir

3 = bajar

El cambio se realiza directamente sobre: posicionAvion.y mediante:

posicionAvion.y += ((teclas[GLFW_KEY_9] ? 1.0f : 0.0f) - (teclas[GLFW_KEY_3] ? 1.0f : 0.0f)) * 3.0f * dtRover;

Además, el avión puede moverse en X y Z y puede girar alrededor de Y.

Spotlight amarillo apuntando al piso

La luz inferior del avión fue definida de color amarillo:

spotLights[indiceFaroAvion] = SpotLight(1.0f, 1.0f, 0.0f, 0.02f, 1.4f, 0.0f, 0.0f, 0.0f, 0.0f, -1.0f, 0.0f,1.0f, 0.09f, 0.032f, 30.0f);

Su dirección local es: (0,-1,0), por lo que apunta directamente hacia abajo.

La posición y dirección se calculan a partir de padreAvion y después se actualiza mediante:

spotLights[indiceFaroAvion].SetFlash(posicionFaroAvion, direccionFaroAvion);



Figura 4. Spotlight amarillo colocado debajo del avión e iluminando directamente hacia el piso

Ejercicio 3. Spotlight frontal del avión

Además de la luz inferior se agregó una segunda fuente spotlight colocada en la parte frontal del avión.

En el código se define:

spotLights[indiceFaroFrontalAvion] = SpotLight(1.0f, 1.0f, 1.0f,0.05f, 2.0f, 0.0f, 0.0f, 0.0f, 0.0f, 1.0f,1.0f, 0.02f, 0.002f, 35.0f);

Avance y retroceso del avión

Para este movimiento se utilizan:

- T = avanzar
- Y = retroceder

Primero se calcula cuál es el frente actual del avión tomando en cuenta su rotación, esto es mejor que simplemente modificar Z, porque si el avión gira, sigue avanzando hacia donde apunta su nariz.

Por lo tanto, la luz hereda tanto la traslación y la rotación



Figura 5. Spotlight frontal del avión orientado en la misma dirección que la nariz del modelo.

Faro frontal ligado al avión

El faro frontal se coloca cerca de la punta de la nariz y su dirección también deriva de la matriz padre:

Resultado

Se logró implementar iluminación tipo spotlight en dos objetos jerárquicos diferentes. En el Rover, el faro frontal utiliza como referencia la misma matriz padre empleada para desplazar el vehículo. Esto permite que su posición se actualice automáticamente cuando el Rover avanza o retrocede.

En el avión se implementaron dos fuentes independientes: una inferior orientada hacia el piso y otra frontal orientada en la dirección de la nariz. Ambas obtienen su posición y dirección a partir de padreAvion, manteniéndose unidas al modelo durante sus transformaciones.

Finalmente, los tres componentes del avión cuerpo, cara y alas son renderizados utilizando la misma matriz padre, manteniendo la jerarquía del modelo..



Figura 6. Resultado final de la escena con spotlights jerárquicos asociadas al Rover y al avión.

Problemas presentados

Uno de los principales retos fue evitar que las fuentes de iluminación permanecieran en una posición fija mientras los modelos se desplazaban. Para solucionarlo, las posiciones locales de las luces fueron transformadas mediante la misma matriz padre utilizada por cada objeto. Las direcciones se transformaron utilizando la parte matricial correspondiente a la orientación, evitando aplicarles la traslación.

También fue necesario calcular el frente actual del avión para que pudiera avanzar en la dirección hacia la cual se encuentra orientado y no únicamente sobre un eje fijo del escenario. En el Rover se mantuvieron matrices auxiliares para conservar las diferentes ramas de la jerarquía, permitiendo que ruedas, brazo, articulaciones y pinza permanecieran relacionados con el cuerpo principal.

Conclusión:

a) La parte más importante de estos ejercicios fue comprender la diferencia entre la posición de la luz y la dirección de la luz, La posición representa un punto, por lo que se transforma usando una coordenada homogénea, mientras que una dirección representa un vector y debe transformarse sin aplicar la traslación, esto permitió que cada spotlight conserve correctamente su posición relativa y orientación con respecto al objeto padre.
b) La actividad permitió aplicar el concepto de jerarquías no solamente a la geometría de los modelos, sino también a las fuentes de iluminación. Anteriormente las matrices padres se utilizaban para que los componentes de un objeto se desplazaran juntos. En estos ejercicios se utilizó el mismo

principio para vincular una fuente de iluminación al modelo, esto permitió que las luces se comportaran como verdaderos faros integrados a los vehículos y no como fuentes independientes colocadas en coordenadas absolutas del escenario.

c) Conclusión: Al finalizar estos ejercicios se logró integrar diferentes fuentes spotlight a objetos jerárquicos con movimiento. El Rover cuenta con un faro frontal azul que permanece unido al vehículo mientras este avanza y retrocede. El avión cuenta con una luz inferior amarilla orientada hacia el piso que acompaña sus movimientos verticales, además de un segundo faro frontal que conserva la dirección de la nariz durante el avance, retroceso y rotación del modelo. El uso de matrices padre permitió relacionar correctamente las transformaciones de los objetos con las posiciones y direcciones de sus fuentes de iluminación, reforzando el concepto de jerarquías dentro de una escena tridimensional.

Bibliografía

- De Vries, J. (s. f.). Light casters. LearnOpenGL. https://learnopengl.com/Lighting/Light-casters
- De Vries, J. (s. f.). *Multiple lights*. LearnOpenGL. https://learnopengl.com/Lighting/Multiple-lights
- Khronos Group. (s. f.). *Coordinate transformations*. OpenGL Wiki. https://www.khronos.org/opengl/wiki_opengl/index.php/Coordinate_Transformations